



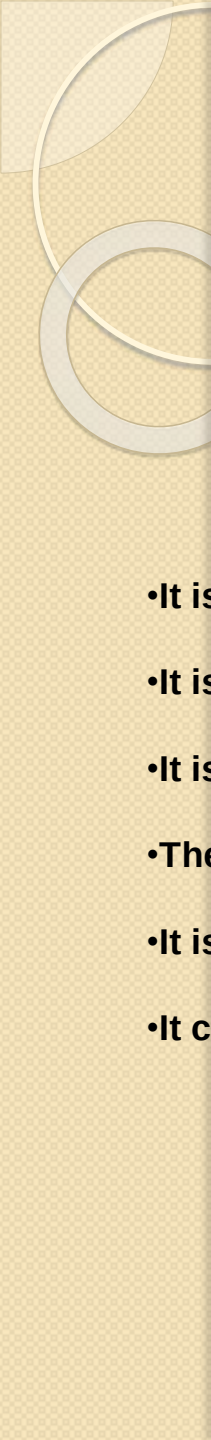
L3 informatique

Course Notes

Software Engineering

Lecture I Introduction

Presented by
Mr KHELIFA N.



Model

- It is a representation of the real world, a process, a machine, or a concept.
- It is an abstraction of reality.
- It is a simplification of a system constructed with a specific purpose.
- The model must be able to answer questions in place of the system being modeled.
- It is a description (of part) of a system written in a well-defined language.
- It can take different forms: diagram, text, ...

Modeling

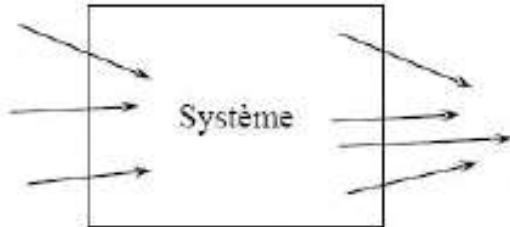
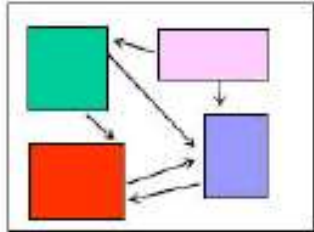
- Modeling is the process of creating a representative model of the real system by using specific methods and languages, such as UML, Petri Nets, Merise, etc.

Graph-based modeling dates back to [Euler, 1735].



Object-Oriented Modeling (OOM)

- Object-Oriented Design (OOD) is the method that leads to software architectures based on the objects of the system rather than on a purely functional decomposition.
It is the structure of the system that gives it its form.
One can start from the domain objects (the basic building blocks) and move upward toward the global system

LA DEMARCHE FONCTIONNELLE	LA DEMARCHE OBJET
Accent mis sur ce que fait le système (ses fonctions)  «Que fait le système ?»	Accent mis sur ce qu'est le système (ses composants) : les objets  «De quoi se compose le système ?»

Modeling language

- A modeling language must define:
 - The semantics of the concepts;
 - A notation for representing concepts;
 - Rules for constructing and using the concepts.
- There are languages with different levels of formalization:
 - **Formal languages (Z, B, VDM):** mostly mathematical, with high expressive power, enabling formal proofs of specifications;
 - **Semi-formal languages (MERISE, UML...):** mostly graphical, with less expressive power but easier to use.
- The software industry has many modeling languages:
 - Adapted to procedural systems (MERISE...);
 - Adapted to real-time systems (ROOM, SADT...);
 - Adapted to object-oriented systems (OMT, Booch, UML...).
- The role of tools (Computer-Aided Software Engineering – CASE tools) is essential for the practical usability of modeling languages.

UML (Unified Modeling Language)

- **UML mainly inherits from the object-oriented methods of Booch (Booch), OMT (Rumbaugh), and OOSE (Jacobson).**
- **Standardized by the OMG (Object Management Group)**
☞ <http://www.omg.org/spec/UML/>
- **Latest version: 2.5.1 (2017)**
- **Standard notation for modeling object-based applications (and component-based applications since version 2).**
- **However, it can also be used in many other design or specification contexts.**
- ***Example: database schema***
- **A language that uses a graphical notation.**

UML

- UML is a language.
- UML is not a method.
- UML is an object modeling language.
- UML has been adopted by all object-oriented methods.
- UML is a standard.
- **UML is a language used to:**
 - **Visualize:** each graphical symbol has a semantics.
 - **Specify:** in a precise, complete, and unambiguous way.
 - **Construct:** classes, relationships, etc.
 - **Document:** the various diagrams, notes, constraints, and requirements

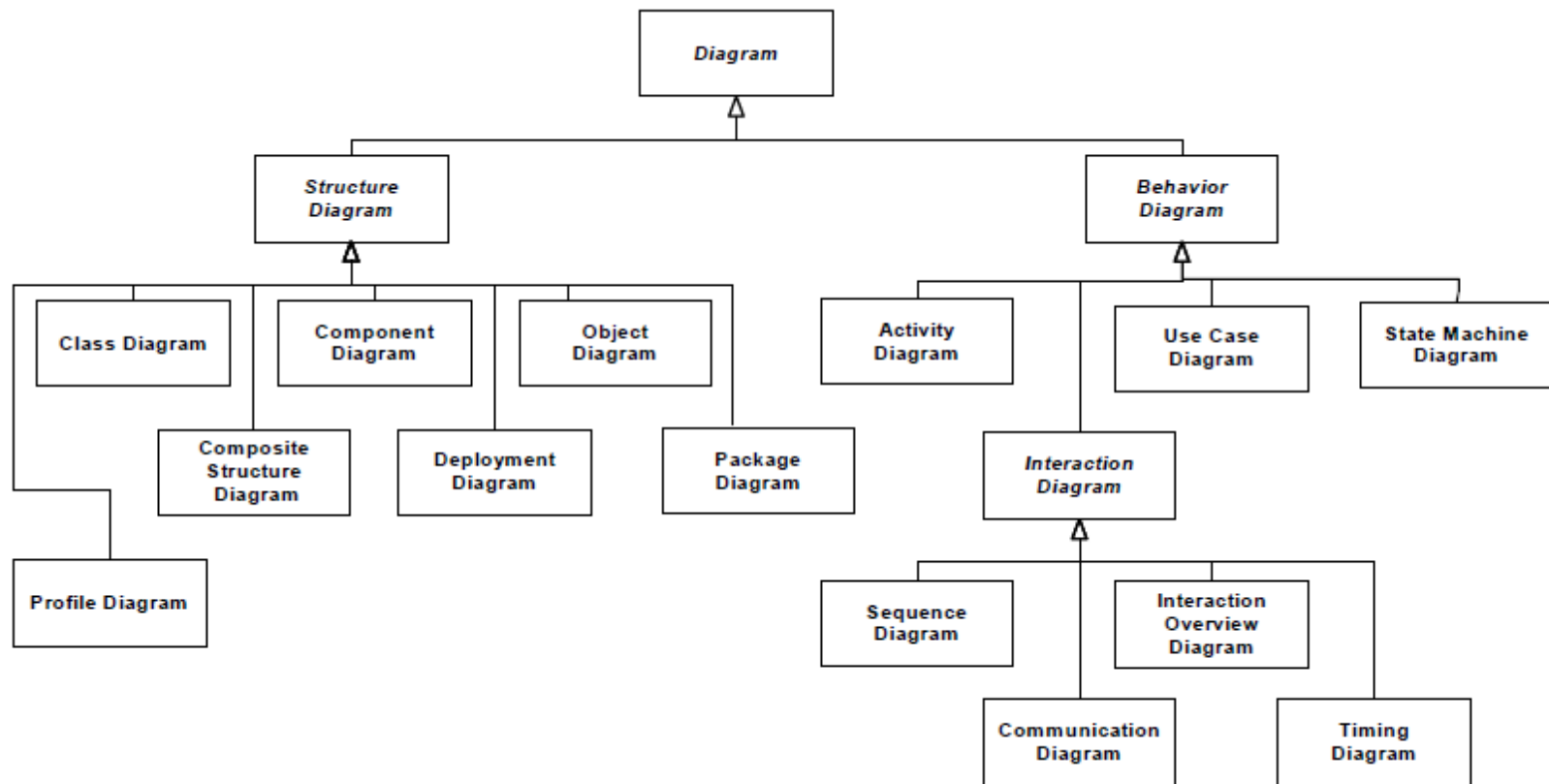
UML

- **UML Building Blocks**
 - **Elements:** the essential abstractions of a model.
 - **Relationships:** the links between the elements.
 - **Diagrams:** meaningful groupings of elements and relationships.

UML in Practice

- **UML Application Domains:**
 - **Enterprise Information Systems**
 - **Banking and Financial Services**
 - **Telecommunications**
 - **Transportation**
 - **Defense and Aerospace**
 - **Scientific Domain**
 - **Web-based Distributed Applications**

UML 2.5.1 Diagrams



Use Case Diagram

- **Use cases provide a means of capturing and describing the needs of the system's actors.**
- They are used as a way to organize software development, particularly for structuring and conducting software testing.
- A use case describes the interaction between actors (users of the case) and the system. The description of the interaction is carried out from the user's perspective.
- The representation of a use case involves the following concepts:
 - **Actor**
 - **Use cases**
 - **Interaction between the actor and the use case**
 - **Relationships between actors**
 - **Relationships between use cases**

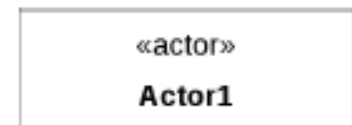
Actor

- **An actor represents a role played by an external entity (human user, hardware device, or another system) that interacts with the system under study.**
- An actor may consult and/or modify the state of the system by sending and/or receiving messages that may carry data.
- There are two types of actors:
 - **Primary actor:** a primary actor obtains an observable result from the system under study.
 - **Secondary actor:** a secondary actor is requested to provide complementary information to the system under study.

- There are two symbols:
 - **Stickman** with the actor's name written below.
 - **Rectangle** stereotyped as «actor» with the actor's name written below.



Nom Acteur



Logical Actors vs. Physical Actors

- **Eliminate “physical” actors as much as possible in favor of “logical” actors:**
 - **An actor is the one who benefits from the use of the system.**
- **This rule makes it possible, in particular, to decouple from interface technologies that may evolve over the course of the project.**

Example

- **The ATM (Automated Teller Machine)**
- An ATM allows any bank card holder to withdraw cash. If the card holder is a client of the bank that owns the ATM, they may also check the balances of their accounts and carry out transfers between their different accounts.
- Transactions are secured, meaning:
 - The ATM queries the bank's Information System (Bank IS) for operations requested by the bank's clients (withdrawals, balance inquiries, and transfers).
 - The ATM queries the Global Bank Card Authorization System (Authorization System) for withdrawals made by non-client cardholders.
- The ATM also requires maintenance operations such as refilling cash, retrieving swallowed cards, etc.
- **Question: Who are the actors of this system?**

Example

- **The ATM (Automated Teller Machine)**
- **An ATM** allows any bank card holder to withdraw cash. If the card holder is a client of the bank that owns **the ATM**, they may also check the balances of their accounts and carry out transfers between their different accounts.
- Transactions are secured, meaning:
 - **The ATM** queries the bank's Information System (Bank IS) for operations requested by the bank's clients (withdrawals, balance inquiries, and transfers).
 - **The ATM** queries the Global Bank Card Authorization System (Authorization System) for withdrawals made by non-client cardholders.
- **The ATM** also requires maintenance operations such as refilling cash, retrieving swallowed cards, etc.
- **Question: Who are the actors of this system?**

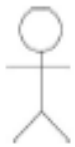
Example

- **The ATM (Automated Teller Machine)**
- **An ATM** allows any bank card holder to withdraw cash. If the card holder is a client of the bank that owns **the ATM**, they may also check the balances of their accounts and carry out transfers between their different accounts.
- Transactions are secured, meaning:
 - **The ATM** queries the bank's Information System (Bank IS) for operations requested by the bank's clients (withdrawals, balance inquiries, and transfers).
 - **The ATM** queries the Global Bank Card Authorization System (Authorization System) for withdrawals made by non-client cardholders.
- **The ATM** also requires maintenance operations such as refilling cash, retrieving swallowed cards, etc.
- **Question: Who are the actors of this system?**

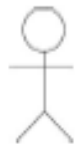
Example

- **The ATM (Automated Teller Machine)**
- **An ATM** allows any bank card holder to withdraw cash. If the card holder is a client of the bank that owns **the ATM**, they may also check the balances of their accounts and carry out transfers between their different accounts.
- Transactions are secured, meaning:
 - **The ATM** queries the bank's Information System (Bank IS) for operations requested by the bank's clients (withdrawals, balance inquiries, and transfers).
 - **The ATM** queries the Global Bank Card Authorization System (Authorization System) for withdrawals made by non-client cardholders.
- **The ATM** also requires maintenance operations such as refilling cash, retrieving swallowed cards, etc.
- **Question: Who are the actors of this system?**

Exemple



Cardholder



Client



Maintenance
operator



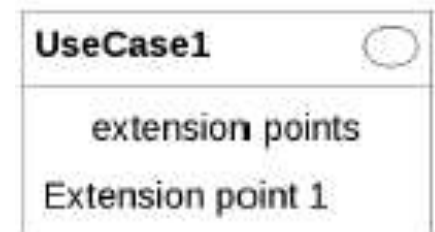
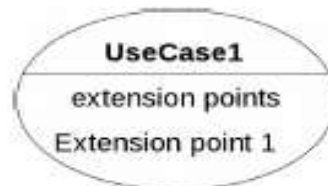
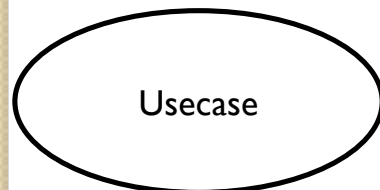
Bank I.S



Auto I,S

Uses cases

- **Represents an expected behavior of the system as a whole without imposing how this behavior is implemented, from the actor's perspective.**
- **It makes it possible to describe what the future system must do.**
- **It represents a set of scenarios carried out by the system that produce an observable result.**
- **Name use cases with an infinitive verb followed by an object, from the actor's point of view (and not from the system's point of view).**



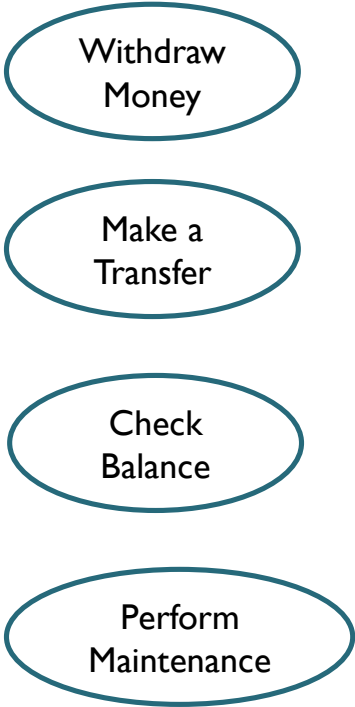
Example

- **The ATM (Automated Teller Machine)**
- **An ATM** allows any bank card holder to withdraw cash. If the card holder is a client of the bank that owns **the ATM**, they may also check the balances of their accounts and carry out transfers between their different accounts.
- Transactions are secured, meaning:
 - **The ATM** queries the bank's Information System (Bank IS) for operations requested by the bank's clients (withdrawals, balance inquiries, and transfers).
 - **The ATM** queries the Global Bank Card Authorization System (Authorization System) for withdrawals made by non-client cardholders.
- **The ATM** also requires maintenance operations such as refilling cash, retrieving swallowed cards, etc.
- **Question: Who are the actors of this system?**

Example

- **The ATM (Automated Teller Machine)**
- **An ATM** allows any bank card holder to withdraw cash. If the card holder is a client of the bank that owns **the ATM**, they may also check the balances of their accounts and carry out transfers between their different accounts.
- Transactions are secured, meaning:
 - **The ATM** queries the bank's Information System (Bank IS) for operations requested by the bank's clients (withdrawals, balance inquiries, and transfers).
 - **The ATM** queries the Global Bank Card Authorization System (Authorization System) for withdrawals made by non-client cardholders.
- **The ATM** also requires maintenance operations such as refilling cash, retrieving swallowed cards, etc.
- **Question: Who are the actors of this system?**

Example



Withdraw
Money

Make a
Transfer

Check
Balance

Perform
Maintenance

Relationship between Actor and Use Case

- **Association:**

An association relationship is a communication path between an actor and a use case (participates in) and is represented by a solid line.

- **Multiplicity (optional):**

When an actor can interact several times with a use case, it is possible to add a multiplicity on the association on the use case side.

- The symbol * means "many."
- Exactly n is written simply n .
- $n..m$ means between n and m , etc.
- Specifying a multiplicity on a relationship does not necessarily imply that the use cases are executed at the same time.

$n..m$

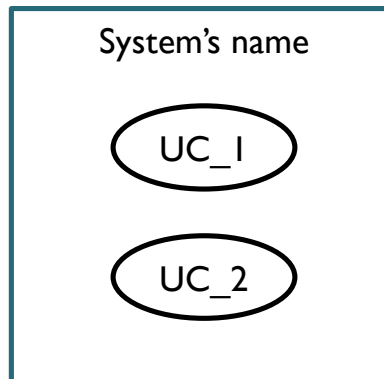
Example



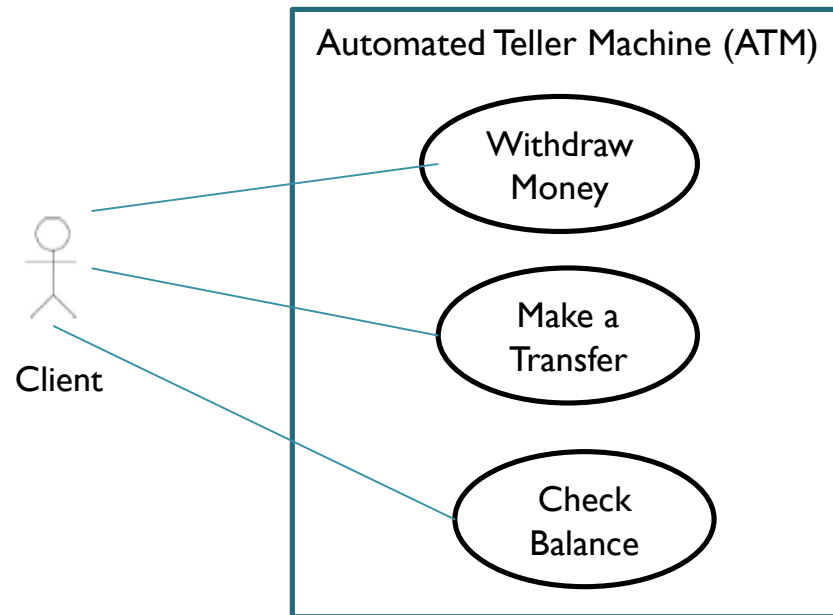
A client can perform several transfers.

System Boundary

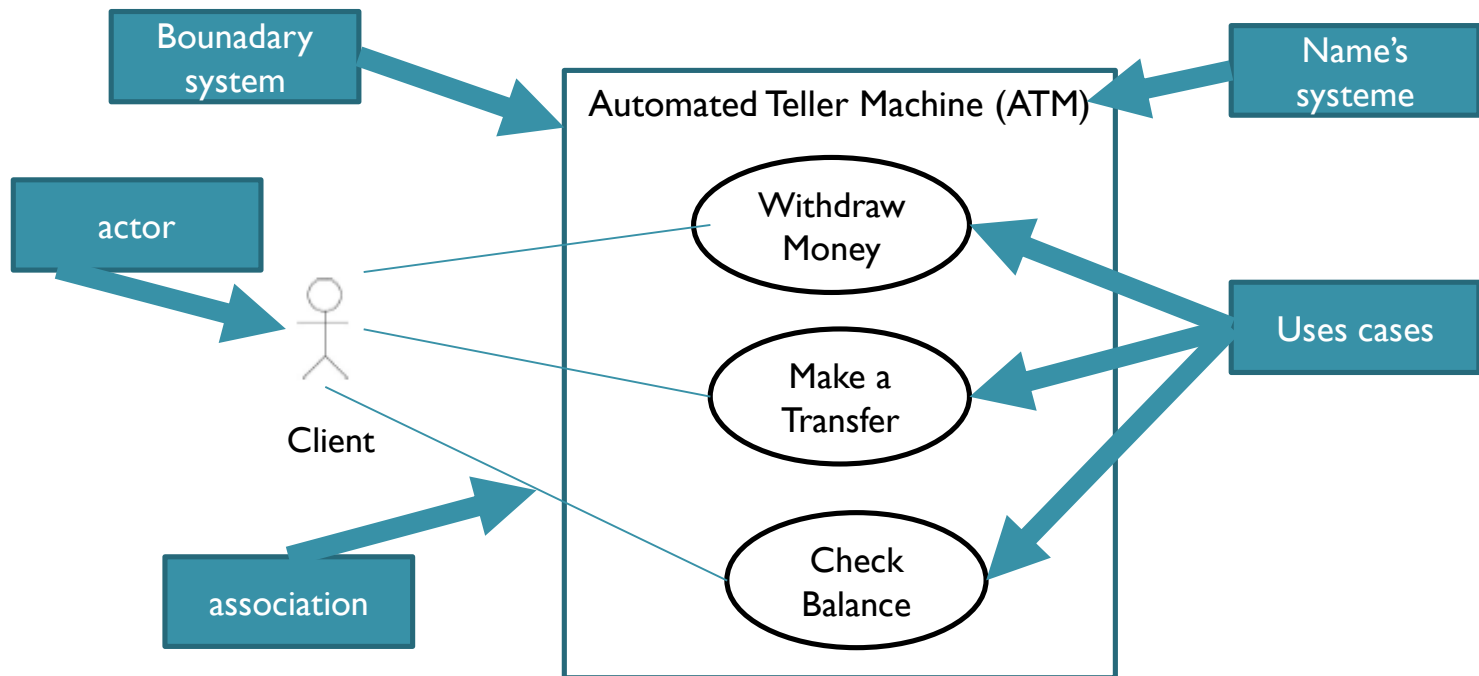
- **The system is a set of use cases; it contains the use cases but not the actors.**
The system boundary determines what is considered external or internal to the system.
- **The system must have a name written at the top of a rectangle that contains the use cases.**



Example



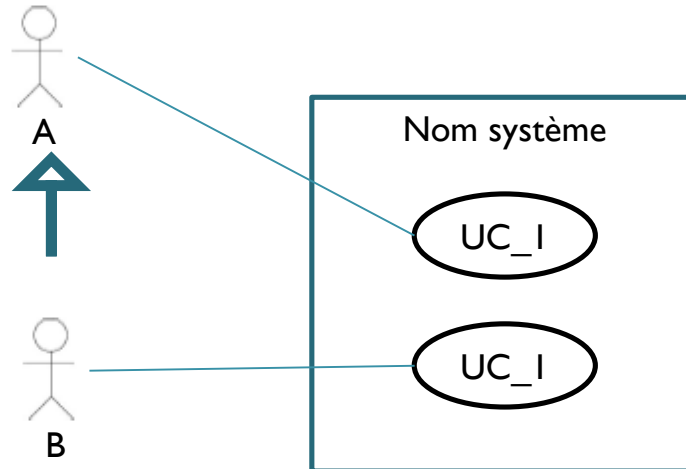
Example



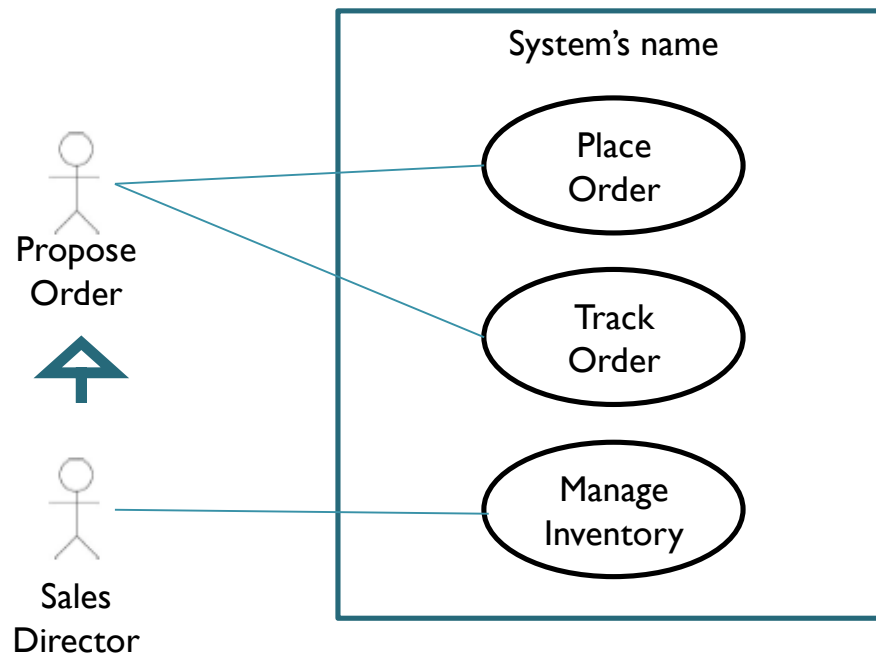
Relationship between Actors

- **Generalization:**

An actor *A* is a generalization of an actor *B* if actor *A* can be substituted by actor *B* (all the use cases accessible to *A* are also accessible to *B*, but the reverse is not true).



Example



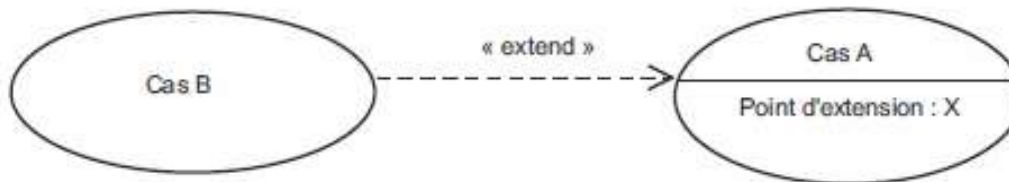


Relationships between Use Cases

- Extension (Extend)
- Inclusion (Include)
- Generalization

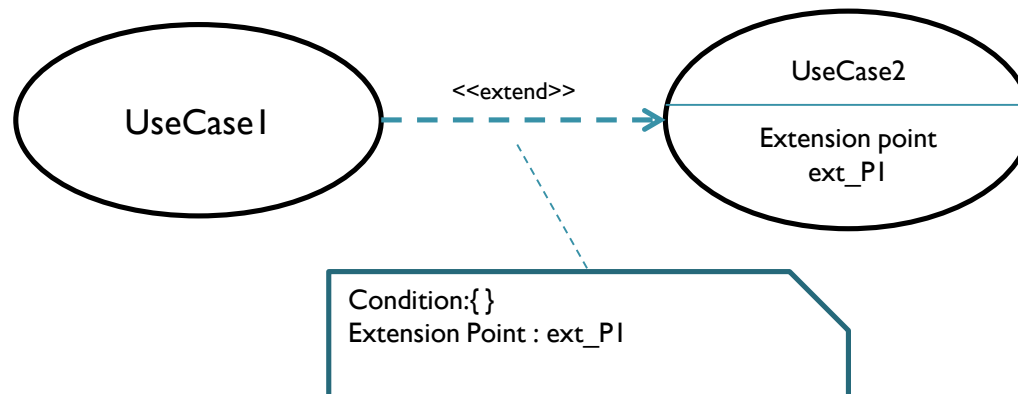
extend

- Extend is intended to be used when there is additional behavior that must be added—possibly conditionally—to the behavior defined in one or more use cases.
The extension can occur at a specific point of the extended use case. This point is called the *extension point*.

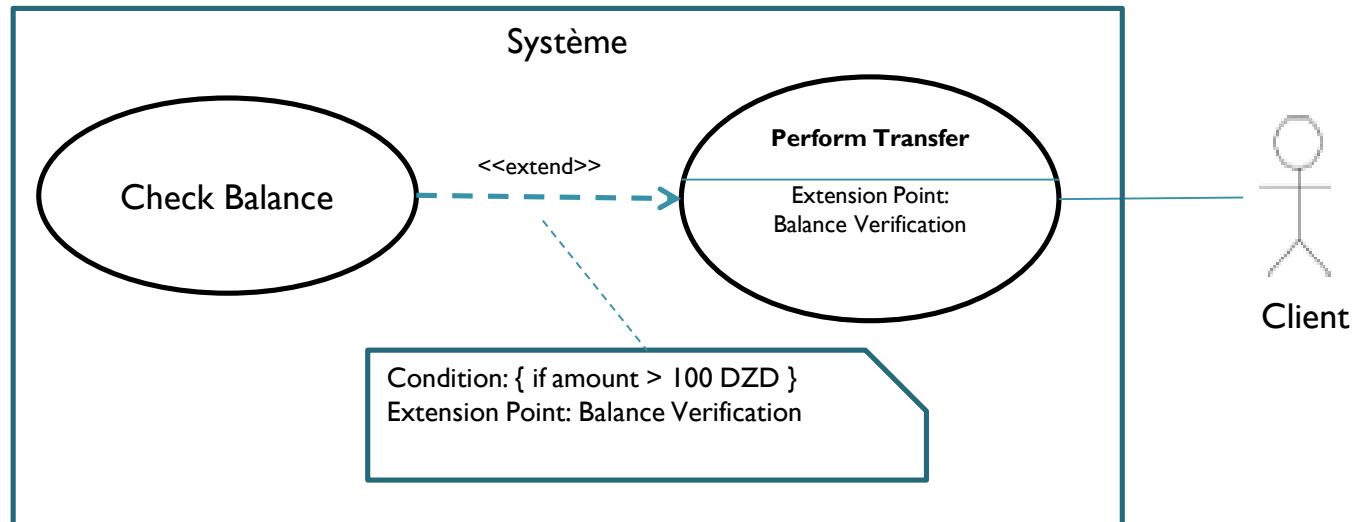


Extend

- The extension point has a name, which appears in a compartment of the extended use case under the heading *extension point*.
An extension is often subject to a condition.
Graphically, the condition is expressed in the form of a note

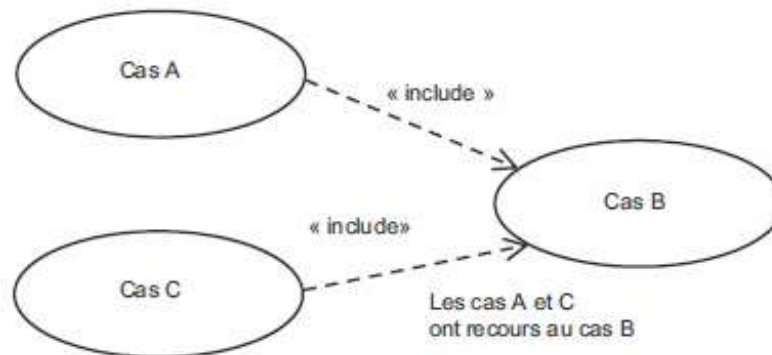


Exemple

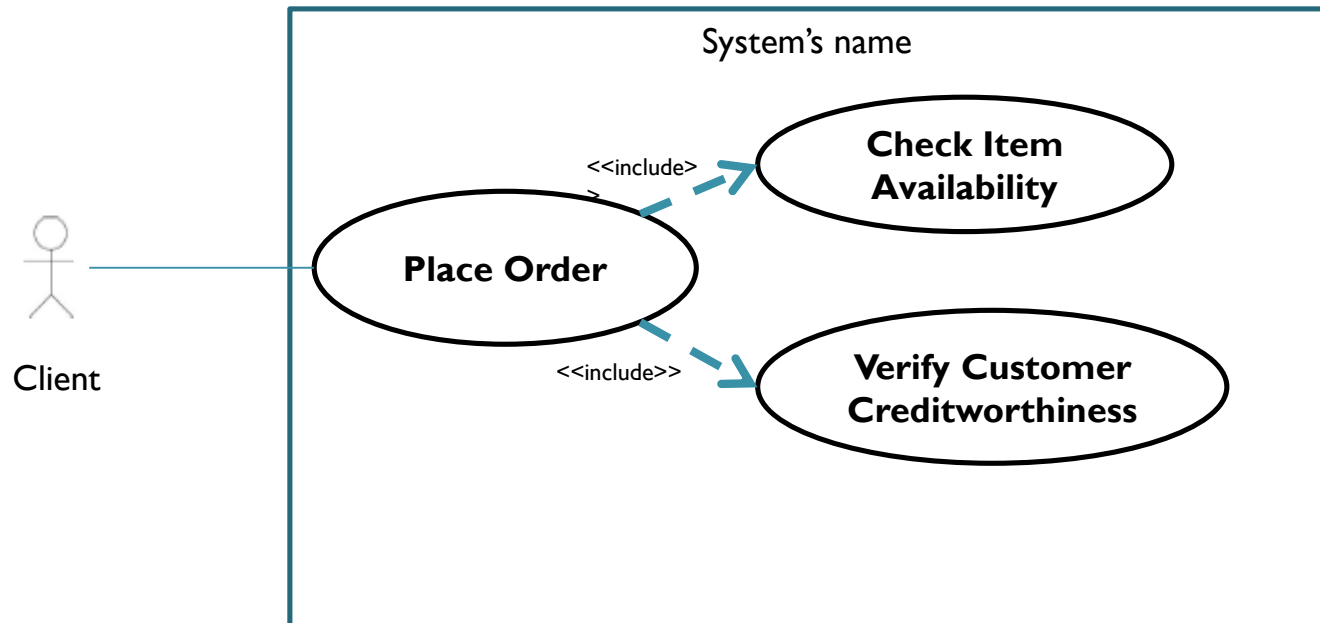


Include

- A use case A includes a use case B if the behavior described by use case A incorporates the behavior of use case B:
use case A depends on B.
- When A is invoked, B is necessarily invoked as a part of A.
This dependency is symbolized by the stereotype <<include>>

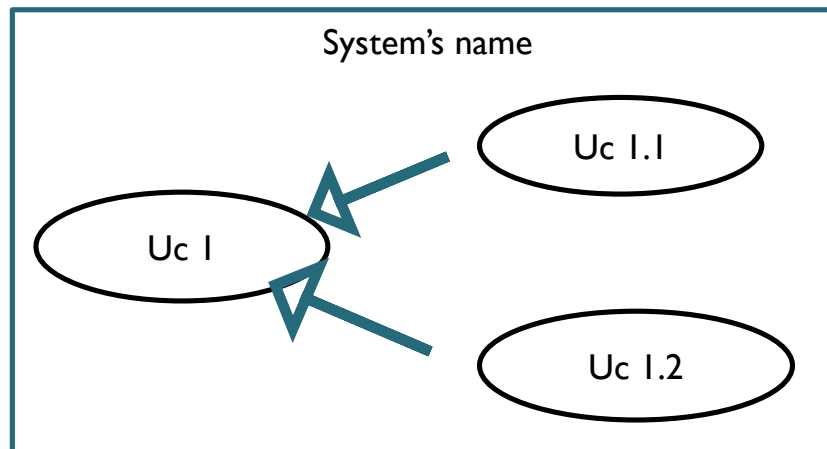


Example

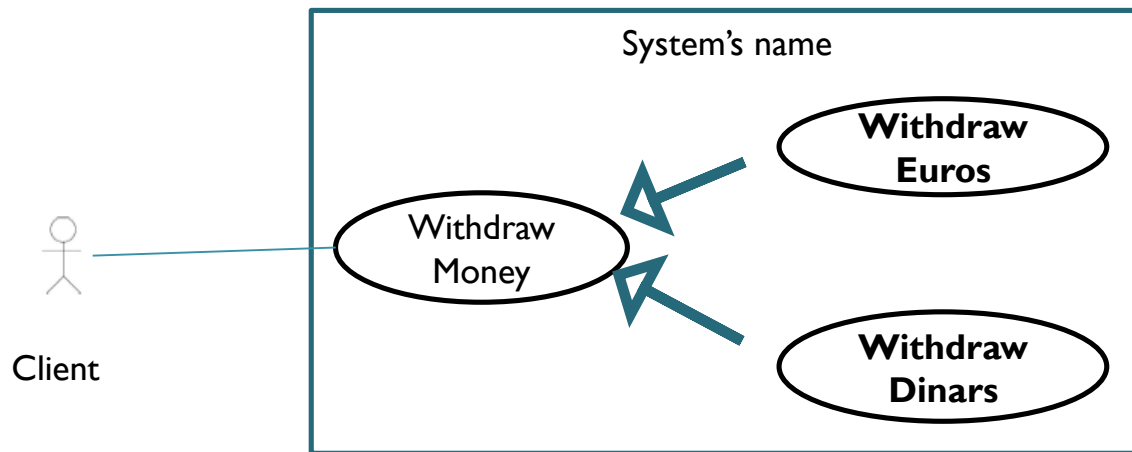


Generalization

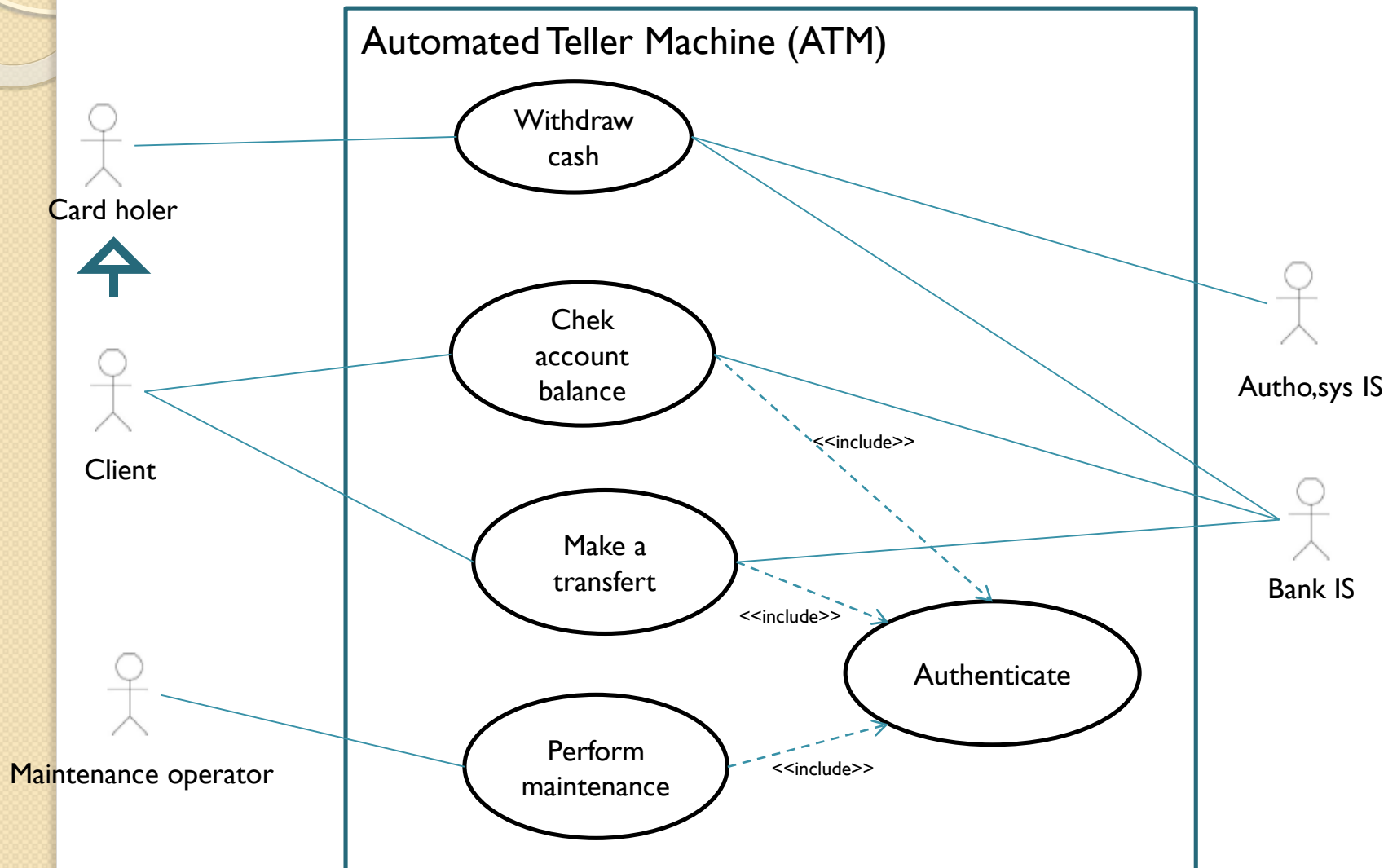
- A use case A is a generalization of a use case B if B is a special case of A. This generalization/specialization relationship is expressed by the concept of inheritance in object-oriented languages.



Example

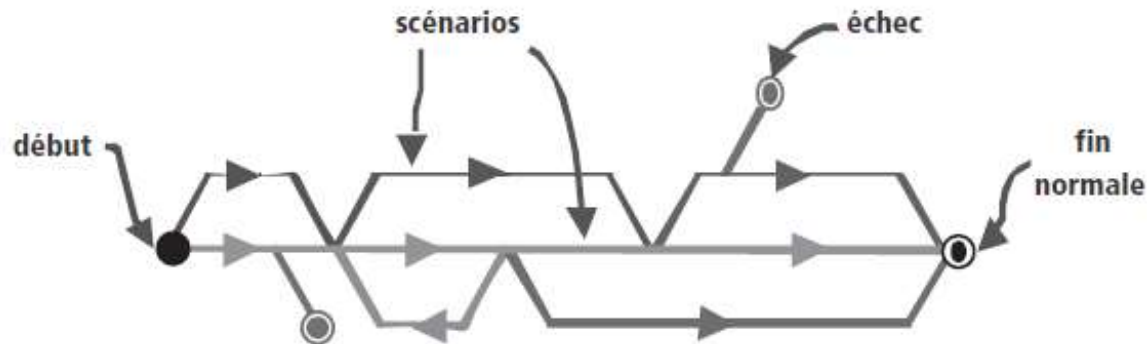


Example



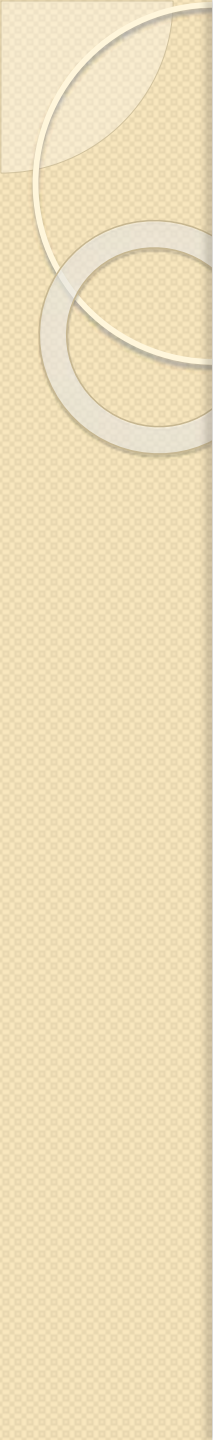
Scenario

- Represents a specific sequence of events executed from the beginning to the end of the use case.
The sequence is the unit of description of the action flow.
A use case generally contains one main (nominal) scenario and several alternative or error scenarios.



Textual Description

- **Each use case can be detailed through a non-standardized textual description as follows:**
 - **Identification**
 - Use Case Name:
 - Goal:
 - Primary Actors:
 - Secondary Actors:
 - Date:
 - Responsible:
 - Version:
 - **Sequencing**
 - Preconditions
 - Main Flow (Nominal Sequence)
 - Alternative Flow
 - Error Flow
 - Postconditions
 - **Optional Sections**
 - Non-functional Constraints





L3 informatique

Notes de Cours

Génie logiciel

Cours I

Introduction

**Présenté par
Mr KHELIFA N.**

Modèle

- Est une **représentation** du monde réel, processus, machine ou concept.
- Est une **abstraction** de la réalité
- Est une **simplification** d'un système construit dans une intention particulière. Le modèle doit pouvoir **répondre à des questions** en lieux et place du système modélisé.
- Est une **description** (d'une partie) d'un système écrite dans **un langage** bien défini.
- Peut prendre différentes formes: schéma, texte, ...

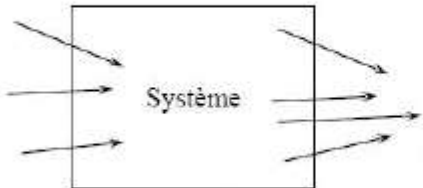
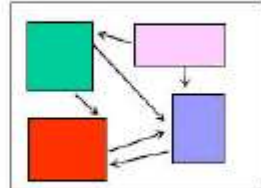
Modélisation

- La modélisation est le processus de création d'un modèle représentatif du système réel en utilisant des méthodes et des langages spécifiques à l'exemple de l'UML, Réseaux de pétri, Merise, etc.
- La modélisation par les graphes date de [Euler 1735]



Modélisation Orientée Objet

- La Conception Orientée Objet (COO) est la méthode qui conduit à des architectures logicielles fondées sur les objets du système, plutôt que sur une décomposition fonctionnelle.
- C'est la structure du système lui donne sa forme.
- On peut partir des objets du domaine (briques de base) et remonter vers le système globale

LA DEMARCHE FONCTIONNELLE	LA DEMARCHE OBJET
Accent mis sur ce que fait le système (ses fonctions)  <i>«Que fait le système ?»</i>	Accent mis sur ce qu'est le système (ses composants) : les objets  <i>«De quoi se compose le système ?»</i>

UML (Unified Modeling Language)

- UML hérite principalement des méthodes objets de Booch (Booch), OMT (Rumbaugh) et OOSE (Jacobson)
- Normalisé par l'OMG (Object Management Group)
<http://www.omg.org/spec/UML/>
- Dernière version : 2.5.1 (2017)
- Notation standard pour la modélisation d'applications à base d'objets (et de composants depuis la version 2)
- Mais utilisable dans de nombreux autres contextes de conception ou spécification
- Exemple : schéma de BDD
- Langage utilisant une notation graphique

UML

- UML est un langage
 - UML n'est pas une méthode,
 - UML est un langage de modélisation objet ,
 - UML a été adopté par toutes les méthodes objet,
 - UML est une norme.
- UML est un langage mis pour
 - Visualiser : chaque symbole graphique a une sémantique,
 - Spécifier : de manière précise et complète, sans ambiguïté,
 - Construire : les classes, les relations.....,
 - Documenter : les différents diagrammes, notes, contraintes, exigences

UML

Brique de base d'UML

- **Éléments:** abstraction essentiel a un modèle
- **Relations:** constituent les liens entre les éléments
- **Diagrammes:** des regroupements d'éléments et de relation digne d'intérêt

UML en application

- Les domaines d'utilisation d'UML :
 - Systèmes d'information des entreprises
 - Les Banques et les services financiers
 - Télécommunications
 - Transport
 - Défense et aérospatiale
 - Domaine scientifique
 - Applications distribuées par le WEB

Les diagrammes UML 2.5.1

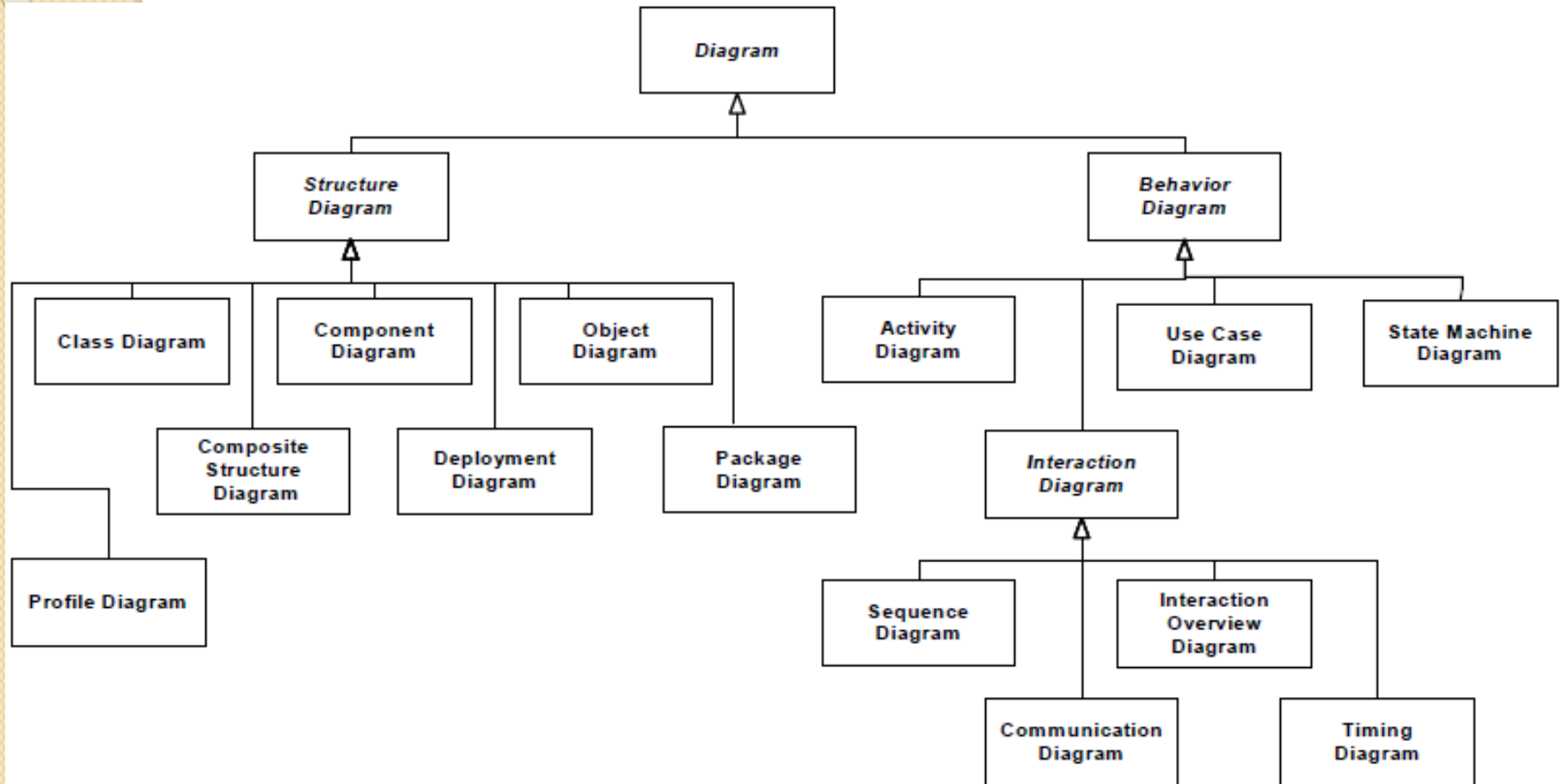


Diagramme de cas d'utilisation

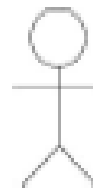
- Les cas d'utilisation constituent un moyen de recueillir et de décrire **les besoins** des acteurs du système.
- Ils sont utilisés comme moyen d'organisation du développement du logiciel, notamment pour la **structuration** et le **déroulement** des tests du logiciel.
- Un cas d'utilisation permet de **décrire l'interaction** entre les acteurs (utilisateurs du cas) et le système. La description de l'interaction est réalisée suivant le point de **vue de l'utilisateur**.
- La représentation d'un cas d'utilisation met en jeu les concepts suivant:
 - l'acteur,
 - les cas d'utilisation
 - l'interaction entre l'acteur et le cas d'utilisation.
 - Relation entre acteurs
 - Relation entre cas d'utilisation

Acteur (Actor)

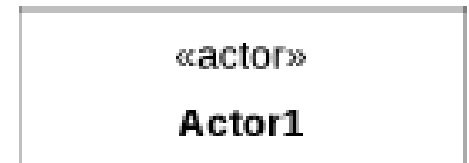
- Un acteur représente un rôle joué par une **entité externe** (utilisateur humain, dispositif matériel, autre système) et qui **interagit** avec le système étudié.
- Un acteur peut **consulter et ou modifier** l'état du système en émettant et ou recevant des messages susceptible de porter des données
- Il existe deux types d'acteur :
 - Un acteur principal: un acteur principale obtient un résultat observable du système étudié.
 - un acteur secondaire est sollicité pour des informations complémentaires pour le système étudié.

Il existe deux symboles :

- Stickman et en dessous le nom de l'acteur
- Rectangle stéréotypé par actor et en dessous le nom d'acteur



Nom Acteur



Acteurs logiques vs acteurs physiques

- Éliminez autant que faire se peut les acteurs « physiques » au profit des acteurs « logiques » :
- l'acteur est celui qui bénéficie de l'utilisation du système.
- Cette règle permet en particulier de s'affranchir des technologies d'interface qui risquent d'évoluer au fil du projet.

Exemple

- Le **DAB** (***D**istributeur **A**utomatique de **B**illet*)
- Un **DAB** permet à tout détenteur de carte bancaire de retirer de l'argent.
- Si le détenteur de carte est un client de la banque propriétaire du DAB, il peut en plus consulter les soldes de ses comptes et effectuer des virements entre ces différents comptes.
- Les transactions sont sécurisées c'est-à-dire :
 - Le **DAB** consulte le Système d'Information de la banque (**S.I. Banque**) pour les opérations que désire effectuer un client de la banque (retraits, consultation soldes et virements).
 - Le **DAB** consulte le Système d'Autorisation Globale Carte Bancaire (**Sys. Auto.**) pour les retraits des porteurs de cartes non clients de la banque.
- Le **DAB** nécessite des opérations de maintenance tel que la recharge en billet, la récupération des cartes avalées, etc.
 - Quel sont les acteurs de ce système

Exemple



Porteur de carte



Client banque



Opérateur de maintenance



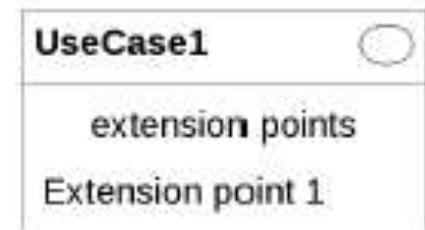
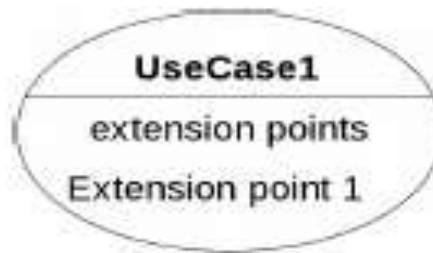
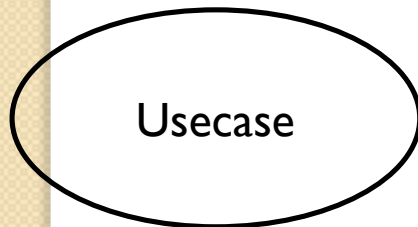
S.I. banque



Sys. Auto

Cas d'utilisation (uses cases)

- Représente un **comportement** attendu du système comme un tout sans imposer le mode de réalisation de ce comportement du point de vue de l'acteur
- Il permet de décrire ce que le future système **doit faire**
- Il représente un ensemble de scénario qui sont réalisés par le système et qui produisent un résultat observable
- Nommez les cas d'utilisation par un verbe à l'infinitif suivi d'un complément, du point de vue de l'acteur (et non pas du point de vue du système).



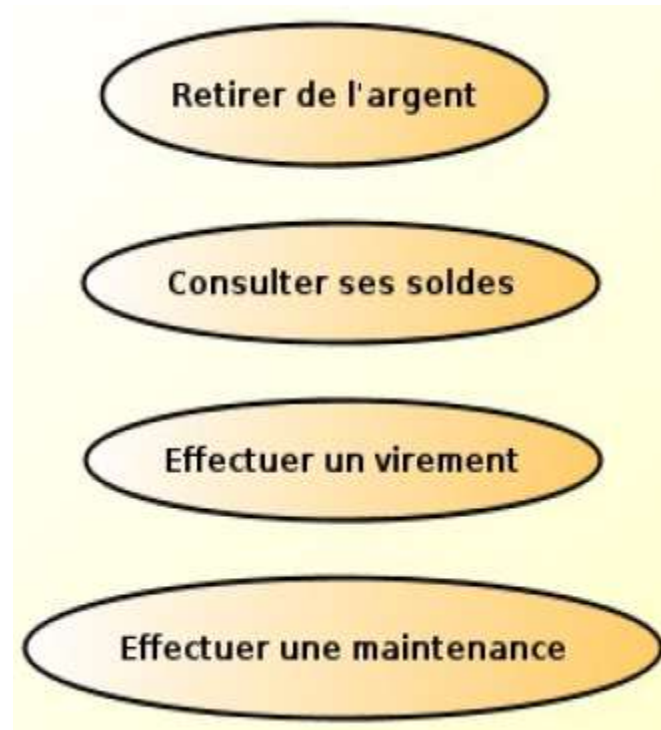
Cas d'utilisation = ensemble de séquences d'actions

- Une erreur fréquente concernant les cas d'utilisation consiste à vouloir descendre trop bas en termes de granularité.
- Le cas d'utilisation ne doit donc pas se réduire systématiquement à une seule séquence, et encore moins à une simple action.

Exemple

- Le **DAB** (***D**istributeur **A**utomatique de **B**illet*)
- Un **DAB** permet à tout détenteur de carte bancaire de retirer de l'argent.
- Si le détenteur de carte est un client de la banque propriétaire du DAB, il peut en plus consulter les soldes de ses comptes et effectuer des virements entre ces différents comptes.
- Les transactions sont sécurisées c'est-à-dire :
 - Le **DAB** consulte le Système d'Information de la banque (**S.I. Banque**) pour les opérations que désire effectuer un client de la banque (retraits, consultation soldes et virements).
 - Le **DAB** consulte le Système d'Autorisation Globale Carte Bancaire (**Sys. Auto.**) pour les retraits des porteurs de cartes non clients de la banque.
- Le **DAB** nécessite des opérations de maintenance tel que la recharge en billet, la récupération des cartes avalées, etc.
 - Quel sont les cas d'utilisation de ce système

Exemple



Relation entre acteur et cas d'utilisation

- L'association:
 - Une relation d'association est chemin de communication entre un acteur et un cas d'utilisation (participe a) et est représenté un trait continu
- Multiplicité (optionnel):
 - Lorsqu'un acteur peut interagir plusieurs fois avec un cas d'utilisation, il est possible d'ajouter une multiplicité sur l'association du côté du cas d'utilisation.
 - Le symbole * signifie plusieurs ,
 - exactement n s'écrit tout simplement n,
 - n..m signifie entre n et m, etc.
 - Préciser une multiplicité sur une relation n'implique pas nécessairement que les cas sont utilisés en même temps.

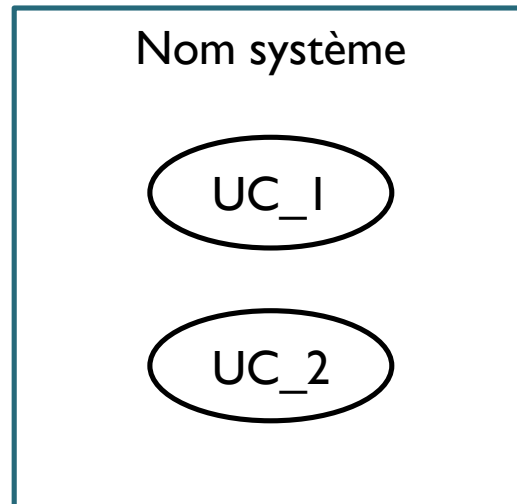
Exemple



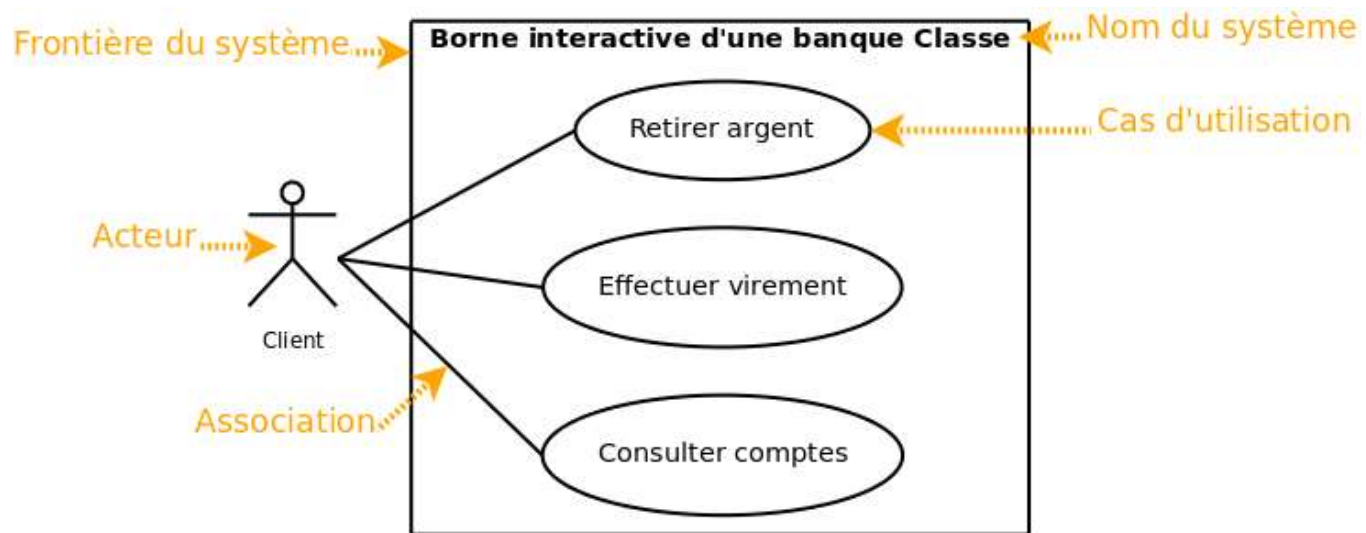
- Un client peut effectuer plusieurs virement

Frontière du système

- Le système est un ensemble de cas d'utilisation, il contient les cas d'utilisation mais pas des acteurs
- la limite système détermine ce qui est considéré comme externe ou interne au système.
- Le système doit avoir un nom écrit en haut d'un rectangle qui contient les cas d'utilisation



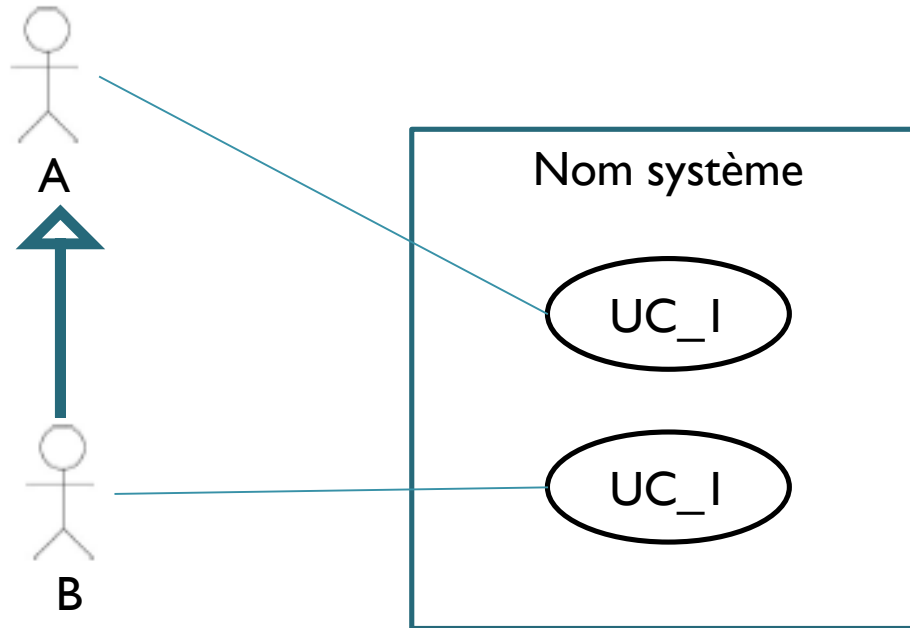
Exemple



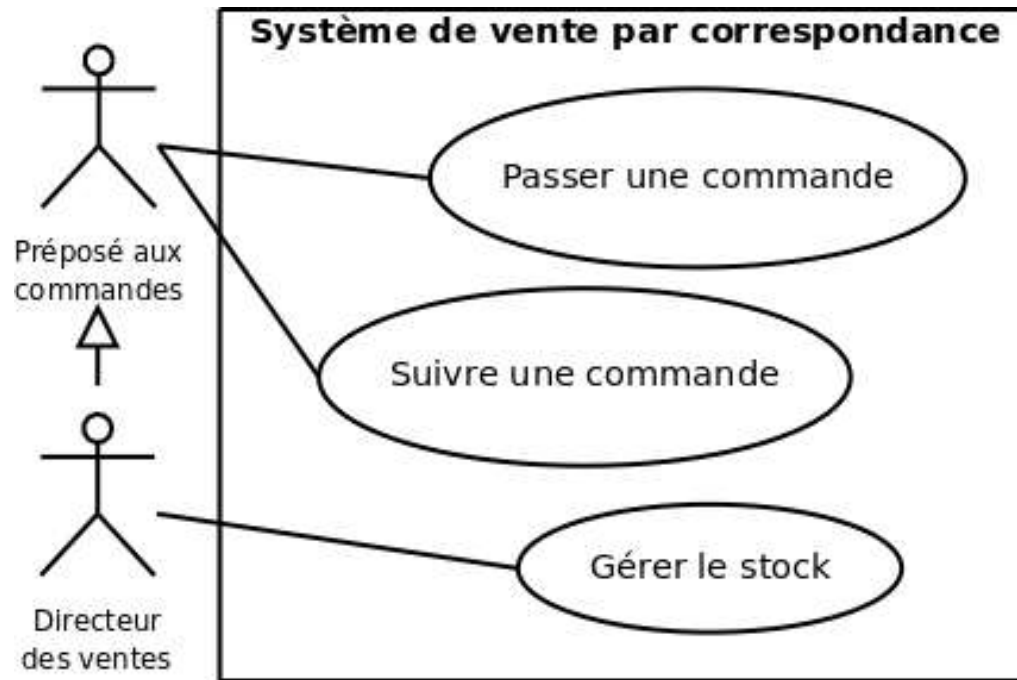
Relation entre acteur

- Généralisation :

- un acteur A est une généralisation d'un acteur B si l'acteur A peut être substitué par l'acteur B (tous les cas d'utilisation accessibles à A le sont aussi à B, mais l'inverse n'est pas vrai)



Exemple



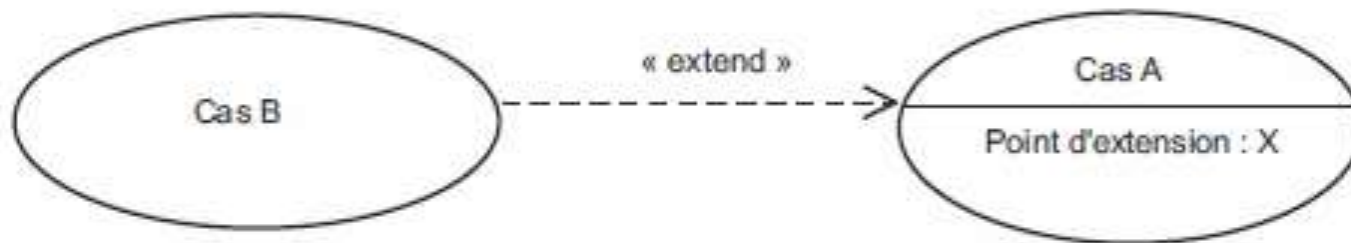
- Le directeur peut Gérer stock, il peut aussi passer commande et suivre commande

Relations entre cas d'utilisation

- Extention (Extend)
- Inclusion (includ)
- Généralisation

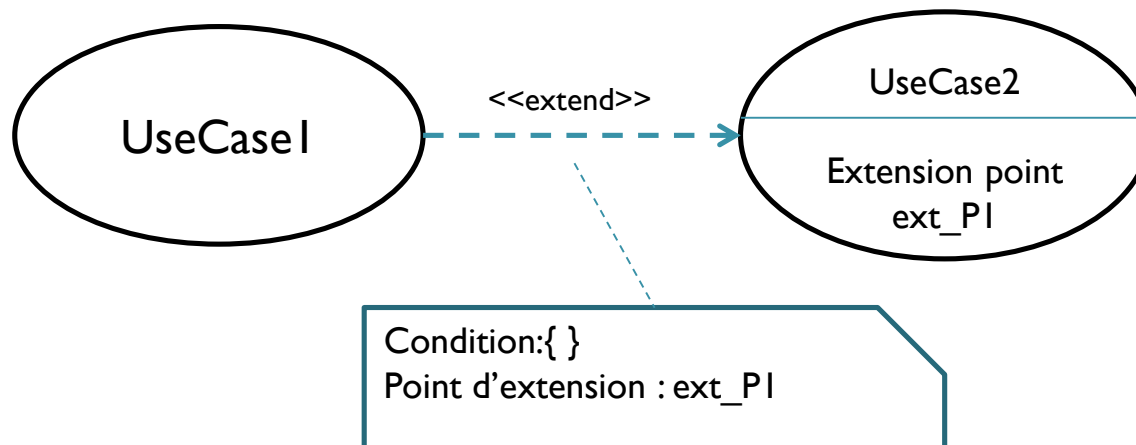
extend

- Extend est destiné à être utilisé lorsqu'il y a un comportement supplémentaire qui doit être ajouté, éventuellement conditionnellement, à un comportement défini dans un ou plusieurs UseCases.
- L'extension peut intervenir à un point précis du cas étendu.
- Ce point s'appelle le point d'extension.

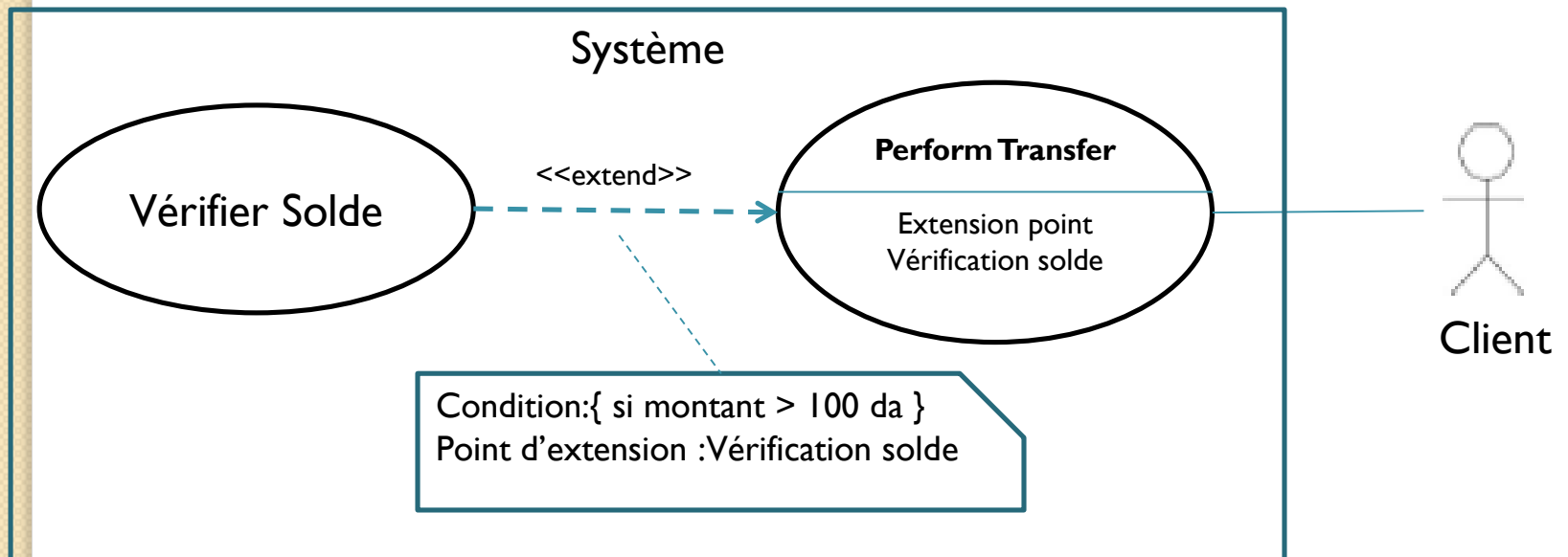


extend

- le point d'extension porte un nom, qui figure dans un compartiment du cas étendu sous la rubrique point d'extension,.
- Une extension est souvent soumise à condition.
- Graphiquement, la condition est exprimée sous la forme d'une note.

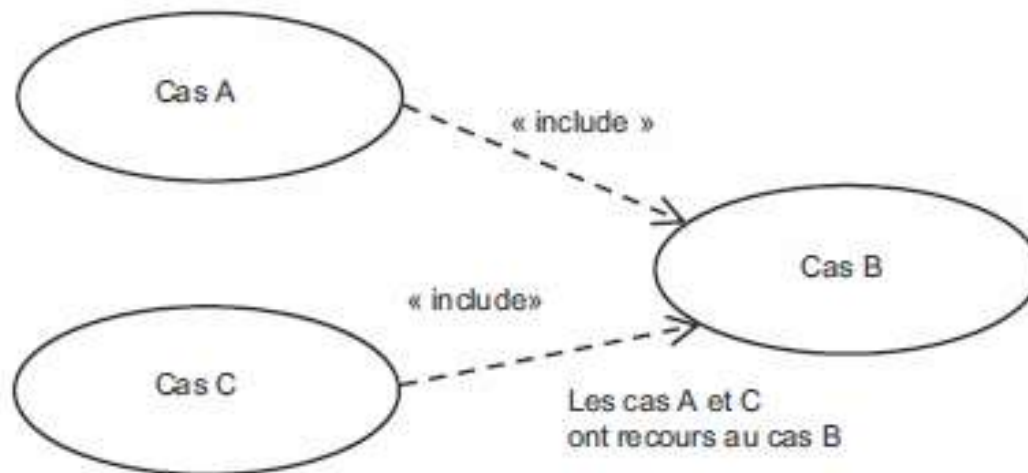


Exemple

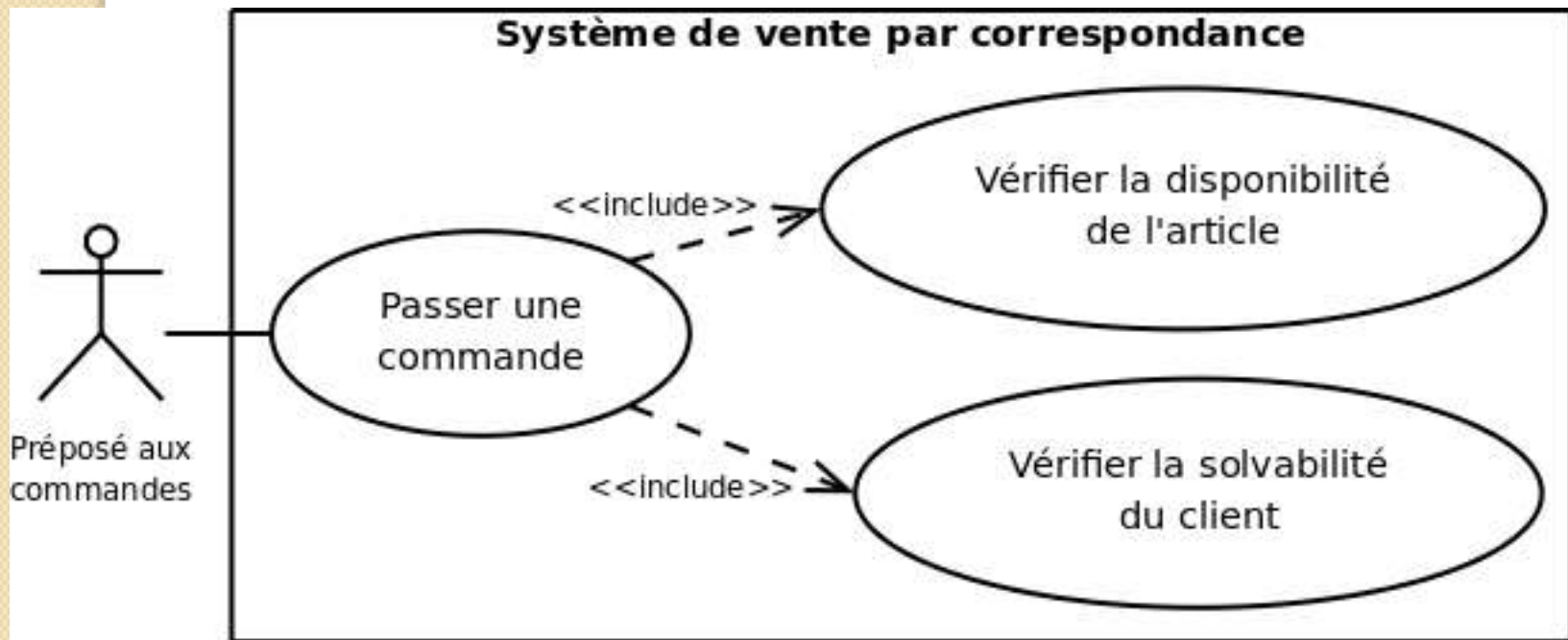


Include

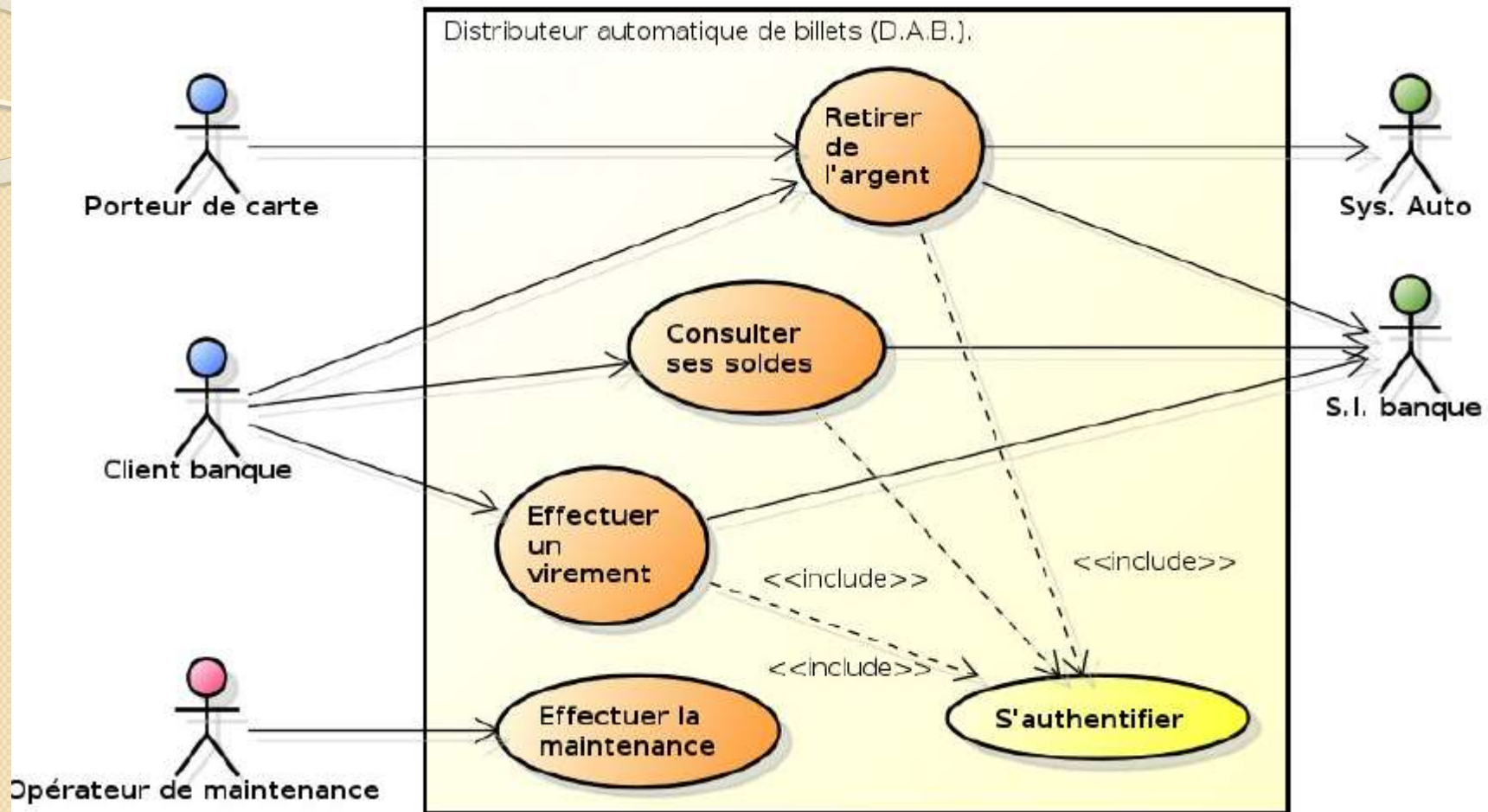
- Un cas A inclut un cas B si le comportement décrit par le cas A inclut le comportement du cas B : le cas A dépend de B.
- Lorsque A est sollicité, B l'est obligatoirement, comme une partie de A.
- Cette dépendance est symbolisée par le stéréotype << include >> .



Exemple I



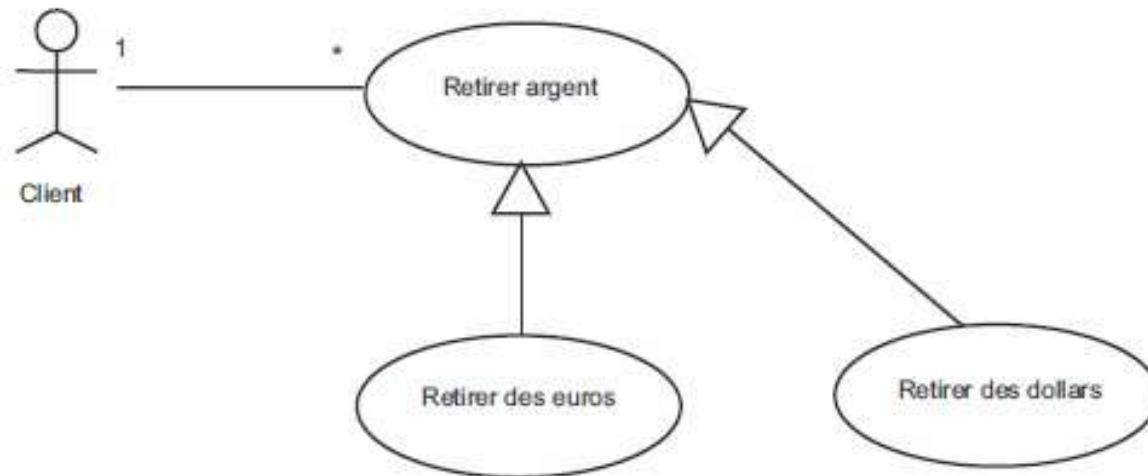
Exemple 2



- l'accès aux informations d'un compte bancaire inclut nécessairement une phase d'authentification avec un identifiant et un mot de passe

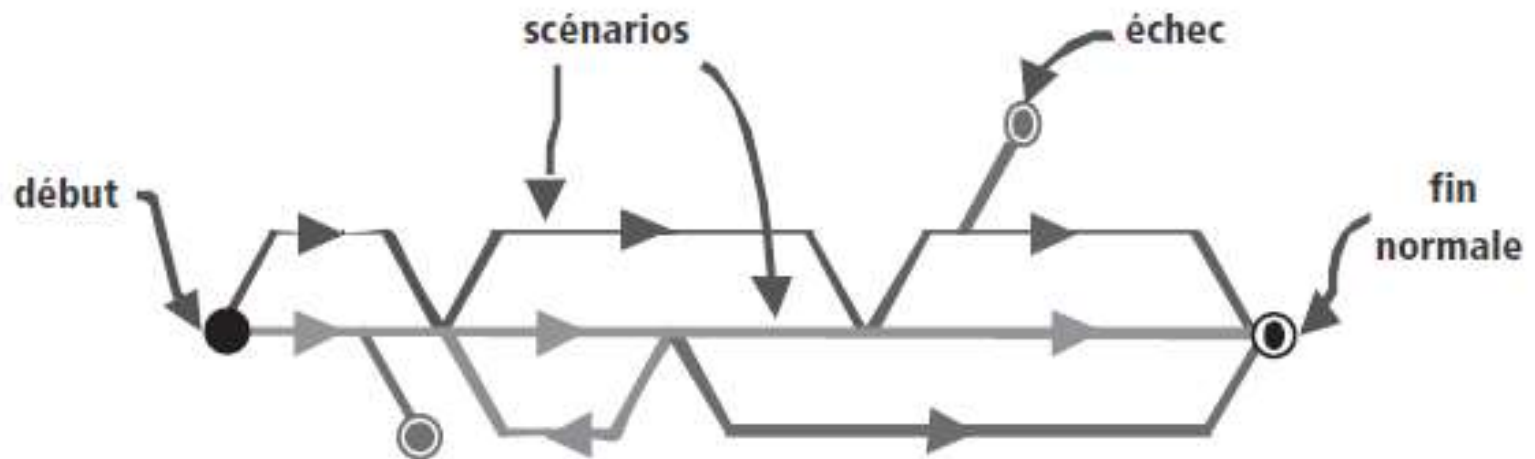
Généralisation

- Un cas A est une généralisation d'un cas B si B est un cas particulier de A.
- Cette relation de généralisation/spécialisation traduit par le concept d'héritage dans les langages orientés objet.



Scénario

- Représente une succession particulière d'enchaînement, s'exécutant de début à la fin du cas d'utilisation
- L'enchaînement étant unité de description de séquence d'action
- Un cas d'utilisation contient généralement un scénario nominale et plusieurs scénarios alternatif ou d'erreur



Description textuelle

- **Chaque cas d'utilisation peut être détaillé a travers une description textuelle non normalisé comme suite :**
- **Identification**
 - Nom du cas :
 - But :
 - Acteurs principal :.
 - Acteurs secondaire :.
 - Date :
 - Responsable :
 - Version :.
- **Séquencement**
 - Pré-conditions
 - Enchaînement nominal
 - Enchaînement alternatif
 - Enchainement d'erreur
 - Post-conditions
- **Rubriques optionnelles**
 - Contraintes non fonctionnelles

